프로젝트 L

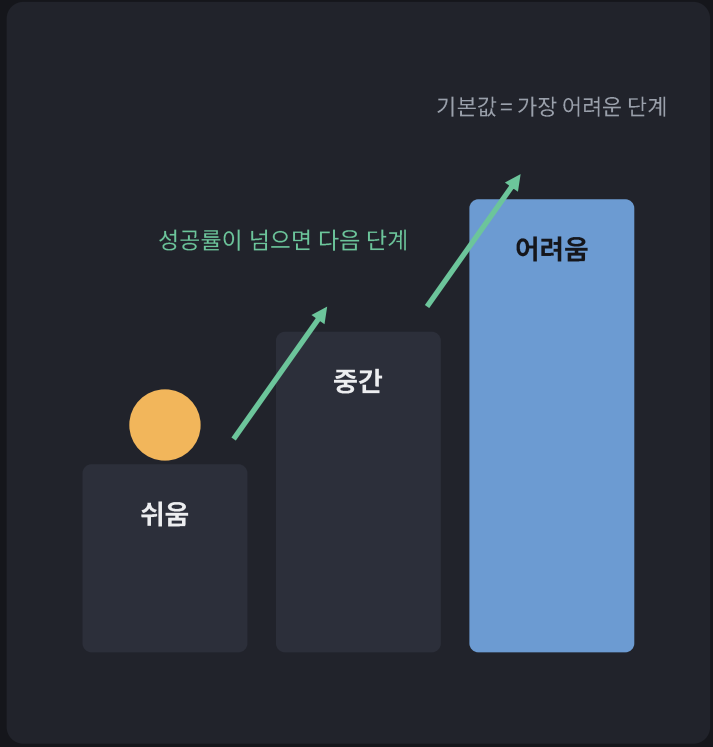
커리큘럼

Curriculum Learning

멀리 있는 목표에 멈춰 서기를, 가까운 목표부터 점점 멀리 난이도를 올리며 학습한다.

난이도 ★★★ · 핵심 개념: 커리큘럼

ML-Agents 강화학습 프로젝트 · 개발 기획서



계단처럼 올라가는 난이도 세 단계

목차

게임을 먼저 설계하고, 그 게임을 강화학습 문제로 옮긴다

01 **프로젝트 개요** — 무엇을 만드는가 — 한 장 요약

02 **게임 디자인** — 컨셉 · 플레이 방식 · 등장 요소 · 사람 조작

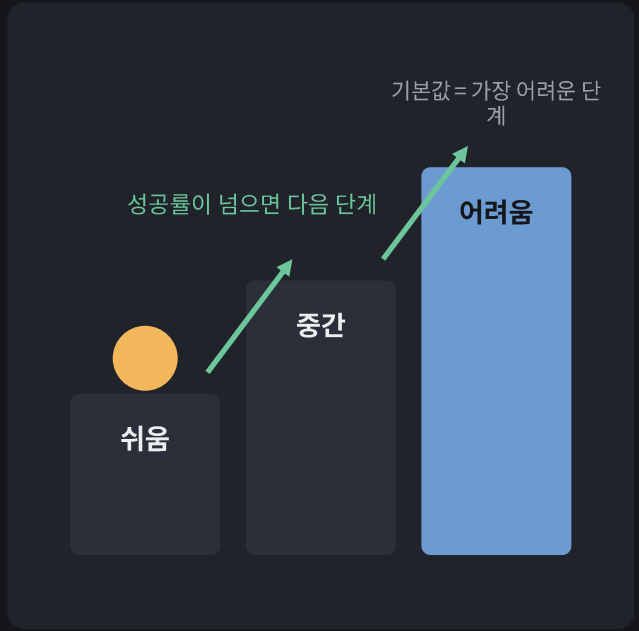
03 **강화학습 설계** — 게임을 학습 문제로 — 관측 · 행동 · 보상 · 에피소드

04 **학습 계획** — 학습 환경 · 개발 단계 · 예상 문제와 대응

프로젝트 개요

멀리 있는 목표에 멈춰 서기를, 가까운 목표부터 점점 멀리 난이도를 올리며 학습한다.

장르	정밀 운전 (B 멈춰서 도착 + 난이도 단계)
플레이어	바닥 위를 구르는 공
목표	가장 먼 단계에서도 목표 원 안에 멈춘다
실패	떨어짐 · 시간 초과
학습 대상	쉬운 판에서 익힌 운전을 먼 거리로 넓히기
새 개념	커리큘럼 (environment_parameters)
예상 학습	바탕 과제의 1~2배 (예상치)



게임 컨셉

이 게임은 무엇이고, 무엇이 잘한 플레이인가

가까운 목표부터 멈추는 법을 익히고, 잘하면 목표가 점점 멀어진다

핵심 재미

실력이 늘면 판이 어려워진다 — 게임의 레벨처럼

잘한 플레이

어느 단계에서든 가속 · 감속 · 정지를 지킨다

못한 플레이

단계가 오르자마자 무너지거나, 첫 단계에 갇힌다

플레이 방식

한 판은 이렇게 흘러간다

판 시작
지금 단계의 거리로 목표

→

상황 보기
거리 · 속도

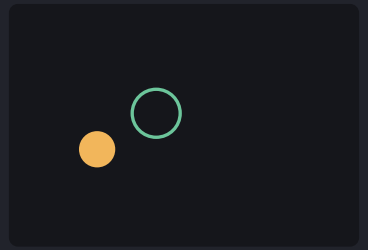
→

조작
밀기 / 브레이크

→

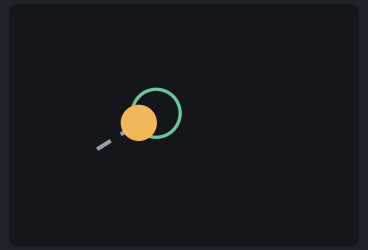
결과
정지 +1 · 단계 판정

한 판 예시



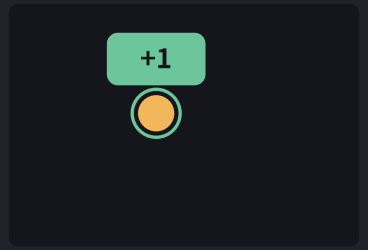
① 단계 읽기

판 시작에 지금 난이도를 받는다



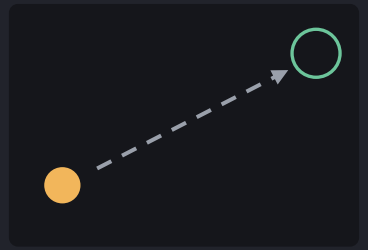
② 이동 · 감속

B 와 같은 운전



③ 정지 +1

원 안에서 멈추면 성공



④ 단계 오름

평균 보상이 문턱을 넘으면

플레이 과정 ① 단계 읽기

판 시작에 지금 난이도를 받는다

① 단계 읽기 | ② 이동 · 감속 | ③ 정지 +1 | ④ 단계 오름



게임에서 일어나는 일

- 파이썬이 정한 '목표 거리'를 읽는다
- 그 거리 안에서 목표 원을 놓는다
- 처음 단계는 가까이 (2)

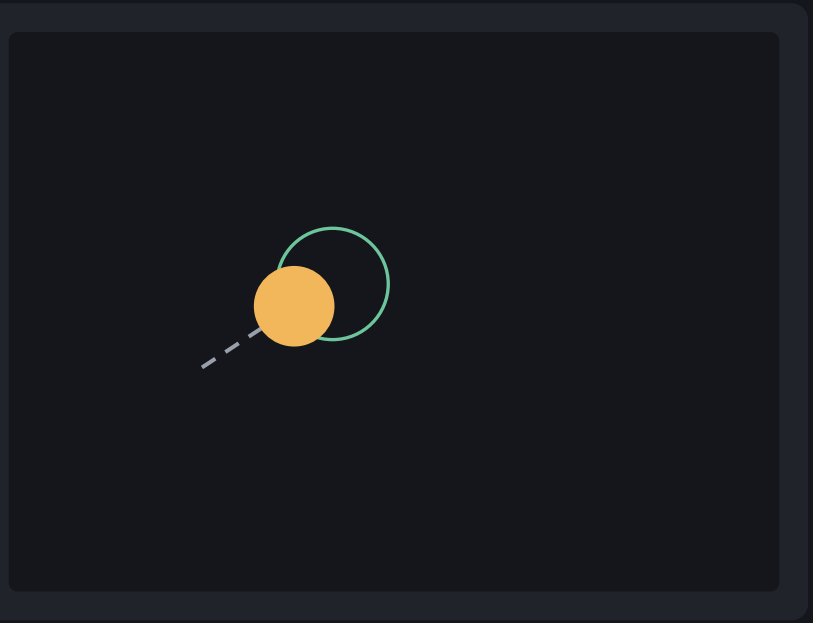
플레이어의 판단 "이번 목표는 가깝다"

관측	행동	보상
목표 – 공 (가까움), 속도 0	첫 결정 전	0 — GetWithDefault("goal_distance")

플레이 과정 ② 이동 · 감속

B 와 같은 운전

① 단계 읽기 | **② 이동 · 감속** | ③ 정지 +1 | ④ 단계 오름



게임에서 일어나는 일

- 목표 쪽으로 밀고
- 가까워지면 반대로 밀어 줄인다
- 보상은 아직 없다

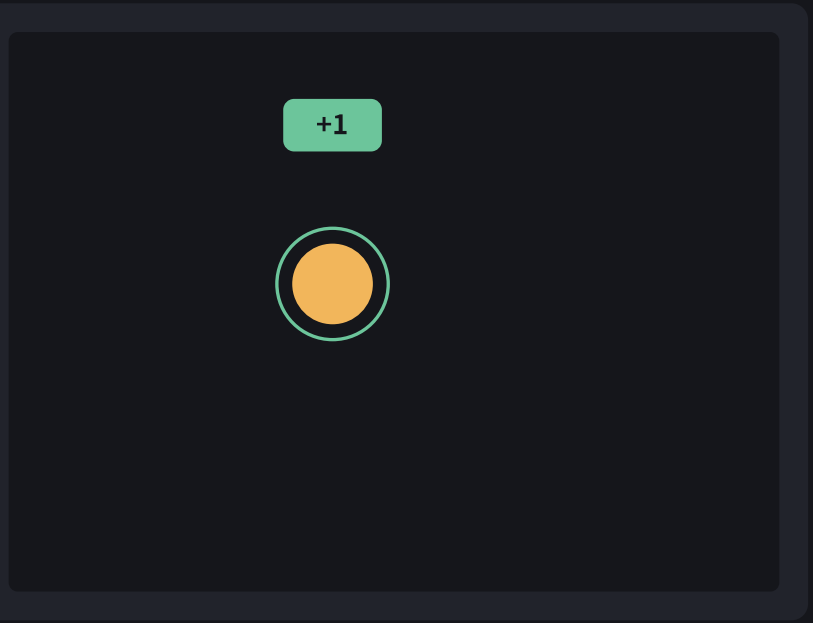
플레이어의 판단 "가까우니 일찍 줄이자"

관측	행동	보상
거리 줄고 속도 커짐	목표 쪽 → 반대 힘	0

플레이 과정 ③ 정지 +1

원 안에서 멈추면 성공

① 단계 읽기 | ② 이동 · 감속 | **③ 정지 +1** | ④ 단계 오름



게임에서 일어나는 일

- 원 안에서 멈춘다
- +1 받고 판 끝
- 성공 기록이 쌓인다

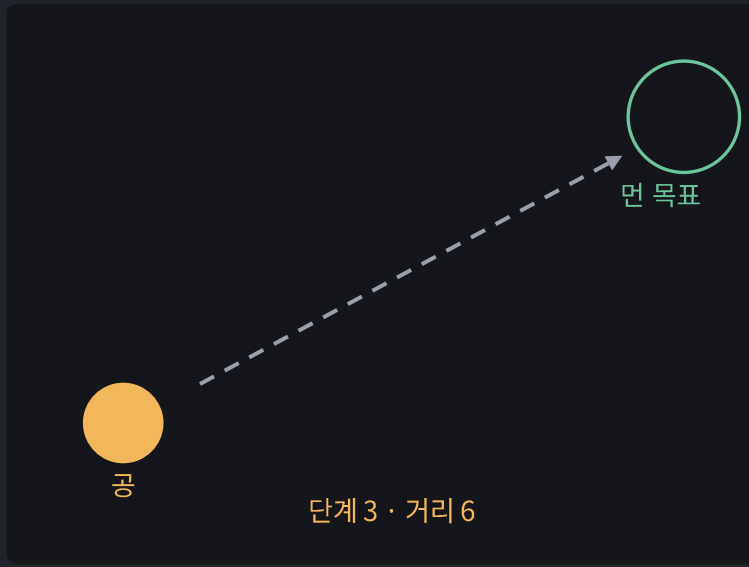
플레이어의 판단 "성공!"

관측	행동	보상
거리 ≈ 0, 속도 ≈ 0	브레이크	+1 — 판 끝

플레이 과정 ④ 단계 오름

평균 보상이 문턱을 넘으면

① 단계 읽기 | ② 이동 · 감속 | ③ 정지 +1 | **④ 단계 오름**



게임에서 일어나는 일

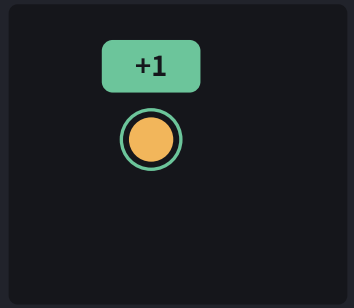
- 최근 판의 평균 보상이 0.8 을 넘는다
- 다음 판부터 목표가 더 멀리 (4 → 6)
- 못 하면 그 단계에 머문다

플레이어의 판단 "목표가 멀어졌다"

관측	행동	보상
목표 – 공 (멀어짐)	—	보상 구조는 그대로 — 거리만 바뀐다

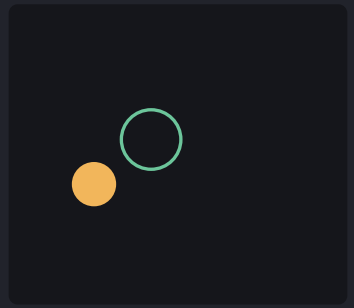
점수와 승패

판의 승패는 B 와 같다 — 커리큘럼은 '다음 판의 난이도'를 정한다



정지 성공

- 원 안 · 속도 < 0.2 → +1
- 성공이 쌓이면 단계 ↑



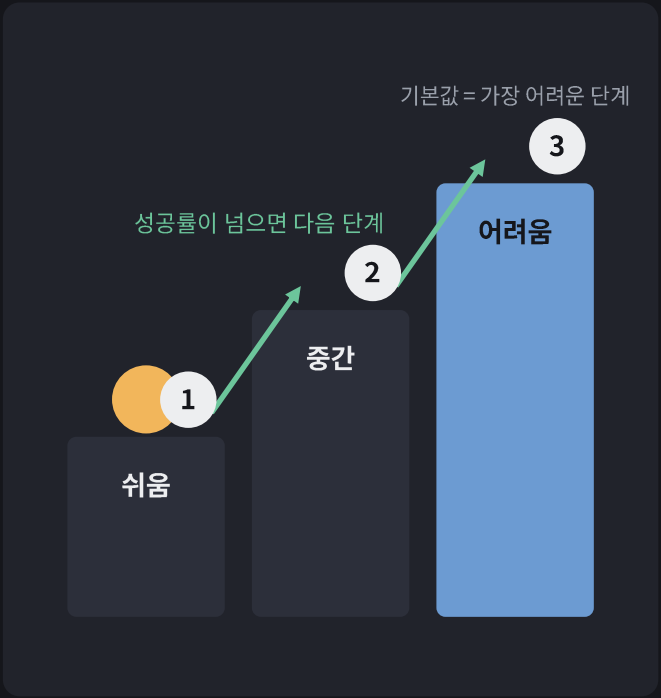
실패

- 떨어짐 · 500 스텝 초과
- 단계가 그대로 머문다

조건	판정 방법 (코드에서)	결과
정지 성공	거리 < 0.7 · 속력 < 0.2	+1 · 판 끝
단계 오름	최근 100판 평균 보상 > 0.8 (yaml)	다음 판부터 목표 거리 ↑
실패	y < 0 · Max Step 500	판 끝

등장 요소

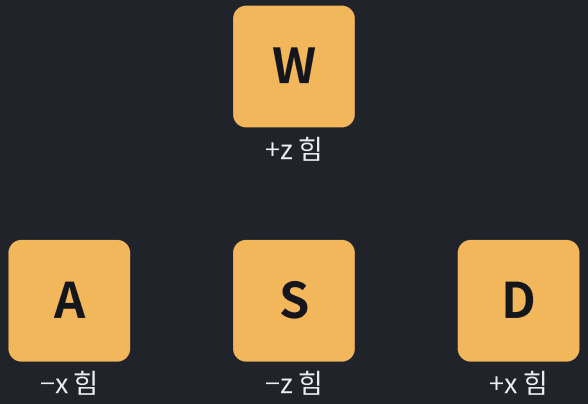
장면에 무엇을 두고, 무엇이 매 판 바뀌는가



#	오브젝트	역할	컴포넌트	매 판
1	단계 1 (Near)	목표 거리 2	yaml lesson — value 2.0	문턱 넘으면 ↑
2	단계 2 (Mid)	목표 거리 4	yaml lesson — value 4.0	문턱 넘으면 ↑
3	단계 3 (Far)	목표 거리 6	yaml lesson — value 6.0 · 코드 기본값	마지막
-	Agent · Target · Floor	B 와 같은 장면	Rigidbody · Agent / 원판 / Plane	거리만 바뀜

사람 조작 (Heuristic)

학습 전에 사람이 먼저 플레이해 본다



왜 먼저 사람이 하나

난이도 값을 손으로 바꿔 각 단계를 직접 풀어 본다. 가장 먼 단계를 사람이 못 하면 단계 설계가 틀린 것

기록해 둘 것

단계별(거리 2 · 4 · 6) 10판 중 성공 수를 적는다

게임 → 강화학습

게임의 각 요소가 강화학습의 무엇이 되는가

강화학습		이 게임에서
Agent	에이전트	공 (B 와 같음)
Environment	환경	바닥 · 목표 + 지금 난이도 값
Observation	관측	B 와 같음 — 난이도는 알려 주지 않는다
Action	행동	B 와 같음 (연속 2)
Reward	보상	B 와 같음 — 원 안 정지 +1
Episode	에피소드	B 와 같음 + 판 시작에 난이도 읽기



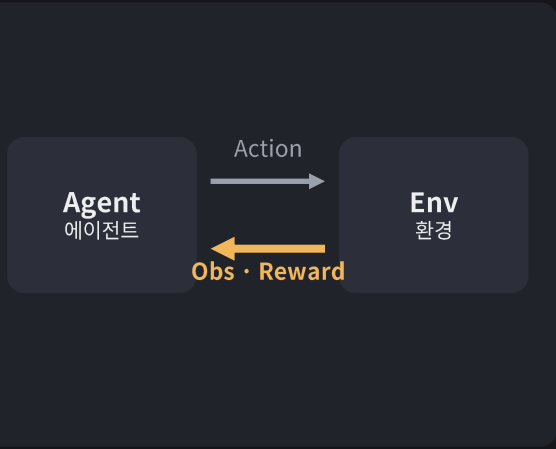
결정할 때마다 관측을 보고 행동을 고르고, 그 결과로 보상을 받는다

관측 설계 Observation

에이전트가 결정할 때마다 보는 숫자

예시 설계 — 바꿔 보고 기록 **= 4칸**

항목	칸	왜 필요한가
목표 – 공 (x, z)	2	B 와 같음
공의 속도 (x, z)	2	B 와 같음



환경 → 에이전트. 칸 수를 Behavior Parameters 의 Space Size 와 맞춘다

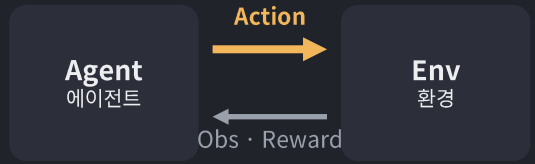
커리큘럼 실험 중에는 관측 · 행동 · 보상을 바꾸지 않는다 — 바꾸면 원인을 모른다

행동 설계 Action

에이전트가 결정할 때마다 고르는 값

예시 설계 — 바꿔 보고 기록 **연속 2**

번호	값	하는 일	키보드
0	−1 ~ +1	x 방향 힘 × 10	A / D
1	−1 ~ +1	z 방향 힘 × 10	W / S

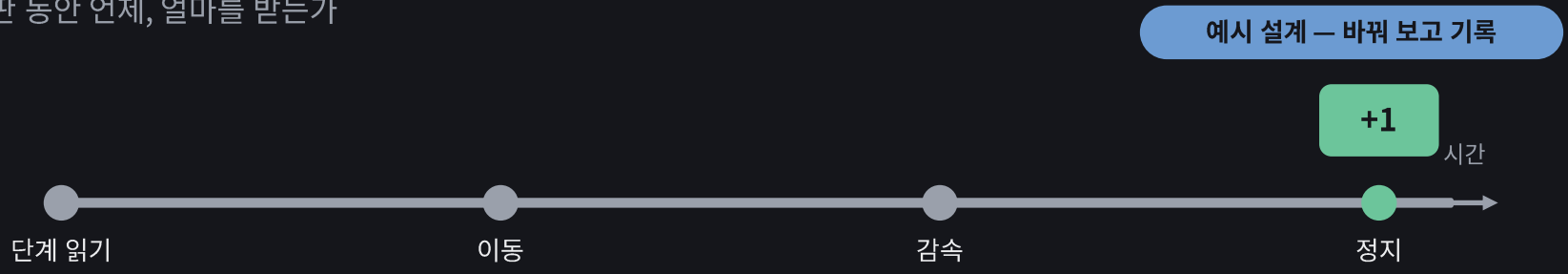


에이전트 → 환경. 행동 순서 = 키보드 순서로 맞추면 Heuristic 이 쉽다

이번 실험의 변수는 '난이도 순서' 하나뿐이다

보상 설계 Reward

한 판 동안 언제, 얼마를 받는가



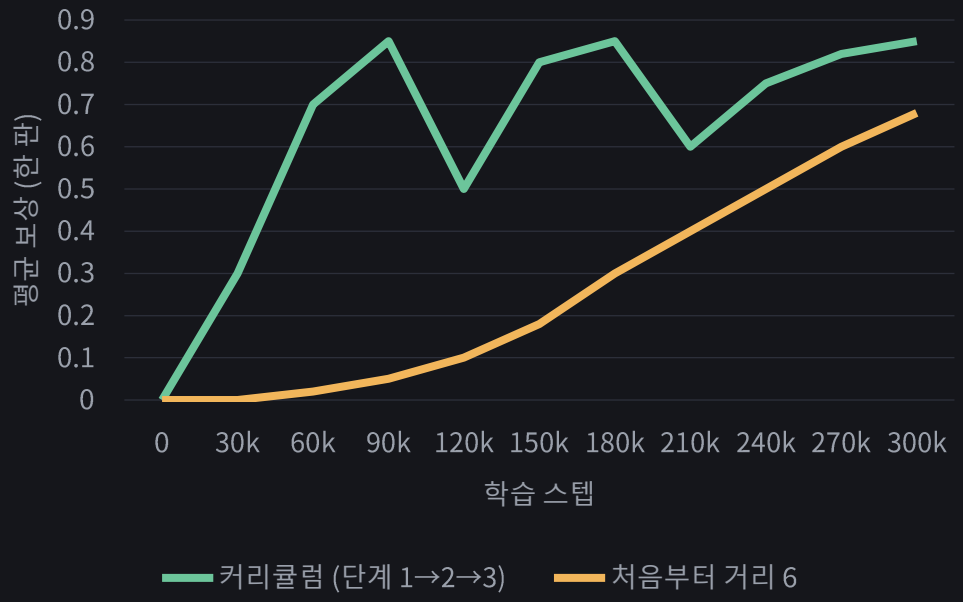
사건	보상	판은?
원 안 정지	+1	판 끝
떨어짐 · 500 스텝	0	판 끝
(단계 오름)	보상 아님	yaml 의 completion_criteria 가 판정

넘어가는 문턱은 쉬운 단계의 실제 평균 보상을 보고 정한다

보상이 만드는 플레이

보상 설계에 따라 어떤 플레이가 높은 점수를 받는가

예시 곡선 — 실측 아님



이 보상이 유도하는 행동

보상은 그대로, 판의 거리만 점점 멀리 → 쉬운 판에서 익힌 감속을 먼 거리로 넓힌다

넣지 않은 보상과 이유

단계가 오를 때 보너스 보상 — 보상 크기가 바뀌어 그래프 비교가 깨진다. 단계는 보상이 아니라 환경이다

에피소드 설계 Episode

한 판은 언제 시작하고, 무엇이 바뀌고, 언제 끝나는가

시작할 때 (OnEpisodeBegin)

goal_distance 읽기 (기본값 6)
그 거리 안에 목표 원
공 무작위, 속도 0

›

진행 (결정마다)

B 와 같음
관측 4칸 → 행동 2개
정지 판정

›

끝 (EndEpisode)

정지 +1 · 떨어짐 · 500 스텝
→ 파이썬이 평균 보상으로
다음 단계 판정

코드의 기본값을 가장 어려운 값(6)으로 — 커리큘럼 없이 돌리면 그대로 비교 실험이 된다

학습 환경 구성

영역을 복제해 한 두뇌가 동시에 여러 판을 경험한다



같은 Behavior Name 의 영역 9개 → 한 두뇌

설정	예시 값	뜻
Behavior Name	StopAtTarget	B 와 같게
environment_parameters	goal_distance	C# 의 이름과 같게
lesson 값	2 → 4 → 6	숫자 하나만
completion_criteria	reward > 0.8, 100판	쉬운 단계를 보고 정한다
max_steps (yaml)	300,000	있음 · 없음 같은 값
예상 시간	30~60분 × 2	예상치 — 직접 잰다

개발 단계

단계마다 만들 것과, 다음으로 넘어가는 기준

1단계
바탕 과제 준비

만들 것

B 멈춰서 도착 완성
목표 거리를
GetWithDefault 로

완료 기준

기본값으로 예전과 똑같이 동작

›

2단계
기준선 학습

만들 것

커리큘럼 없이
거리 6 으로 학습
스텝 수 고정

완료 기준

기준선 그래프 저장

›

3단계
커리큘럼 학습

만들 것

yaml 에 lesson 3개
같은 스텝 수로 학습
Lesson 그래프 확인

완료 기준

단계가 2 → 3 까지 오른다

›

4단계
비교 · 완성

만들 것

거리 6 에서 20판씩
두 모델 평가
그래프 나란히

완료 기준

비교표 · 시연 영상

예상 문제와 대응

학습이 엉뚱하게 갈 때 먼저 볼 곳

단계가 안 넘어간다

원인 문턱이 도달 불가능한 값

대응 쉬운 단계의 실제 평균을 보고 정함

넘어가자마자 무너진다

원인 yaml 커리큘럼은 안 내려온다

대응 min_lesson_length 를 늘리거나 코드로 조절

기본값만 쓴다

원인 C# · yaml 이름 철자 다름

대응 "goal_distance" 를 양쪽에서 확인

한 번에 하나만 바꾸고, 바꾼 이유와 결과를 숫자로 기록한다